

AURIX: Zero-Cost Tech Stack Usage Guide (Backend & Infrastructure)

This guide breaks down exactly **when** and **how** you will use each piece of the technology stack for your backend role. Most importantly, it ensures you can build the entire enterprise-grade architecture using **100% free-tier services and open-source tools**.

🔧 The Free Tech Stack Arsenal

Technology	Purpose in AURIX	Cost Strategy
Node.js & Express.js	Core API Gateway & routing logic	Open-source (Free)
Supabase	DB, Auth, Object Storage, and Vector DB (pgvector)	Generous Free Tier (500MB DB, 1GB Storage, 50k MAU)
Upstash	Redis Message Queue & Rate Limiting	Free Tier (10,000 requests/day max)
Hugging Face	AI Embeddings for the RAG Node	Free Inference API or local Transformers.js
Render / Railway	Web hosting for the Node.js API Gateway	Free Tier (Spins down when idle, perfectly fine for dev)

NPM Packages	multer, node-cron, express-rate-limit	Open-source (Free)
---------------------	---------------------------------------	--------------------



Step-by-Step Usage Guide (From Scratch to Production)

Step 1: Laying the Foundation (The Database & Auth)

When to use: Day 1 of development. **What you use:** Supabase (Free Tier)

- **PostgreSQL Database:** You will create your Supabase account and execute SQL to create the `USERS`, `PROJECTS`, `SCANS`, and `VERIFIED_VULNERABILITIES` tables.
- **Supabase Auth:** You will configure email/password or GitHub login. You use this to issue secure JWTs (JSON Web Tokens) to Bhumika (Frontend) and Divyanshi (VS Code) so they can authenticate their users.

Step 2: Building the API Gateway (The "Traffic Cop")

When to use: When you need to start accepting HTTP requests from the Frontend and VS Code. **What you use:** Node.js, Express.js, & Render

- **Express.js:** You will write routes like `POST /api/scans/github` and `GET /api/scans/{scan_id}`.
- **Hosting:** You will deploy this Express app to **Render** (or Vercel/ Railway) on their free tier so it has a live public URL.

Step 3: Handling Heavy File Uploads Safely

When to use: When Divyanshi (VS Code) needs to send you `.zip` files containing the user's source code. **What you use: Multer (npm) & Supabase Storage (Free Tier)**

- **Multer:** You will use this middleware in Express to parse incoming `multipart/form-data`.
- **Supabase Storage:** Because free hosting providers like Render have tiny hard drives, you cannot save the `.zip` files locally. You will use the `@supabase/supabase-js` SDK to stream the zip files directly into a Supabase Storage Bucket.

Step 4: The Shock Absorber (Message Queuing)

When to use: When 10 users click "Scan" at the same time. The AI takes minutes to run; your API will crash or timeout if you wait for it.

What you use: Upstash Redis (Free Tier)

- **Task Queue:** Instead of waiting for the scan, your Express app will push the scan job (containing the Supabase Storage URL) into an Upstash Redis List (`LPUSH`).
- You then instantly return a `202 Accepted` response to the user. Divyansh's AI Worker will pull jobs from this Upstash Redis queue at its own pace.

Step 5: The AI RAG Upgrade (Vector Search)

When to use: When building the Threat Intelligence database so the AI agents can learn how to fix bugs. **What you use: Supabase pgvector & Hugging Face (Free)**

- **pgvector:** You will run `CREATE EXTENSION vector;` in your Supabase DB. This allows you to store arrays of numbers (embeddings) next to text.
- **Hugging Face (Transformers.js or Free Inference API):** OpenAI embeddings cost money. Instead, you will write a script (`seed_rag.js`) that uses free Hugging Face models (like `all-MiniLM-L6-v2`) to convert OWASP markdown files into vector embeddings for absolutely \$0.

- **Supabase RPC:** You will write a SQL function in Supabase that performs "Cosine Similarity" to quickly find the right vulnerability template when queried by the AI engine.

Step 6: Security & Optimization (Don't hit free tier limits!)

When to use: Nearing production, to ensure your free tiers don't get suspended for overuse. **What you use:** Upstash Redis, `express-rate-limit`, & `node-cron`

- **Rate Limiting:** You will use the `express-rate-limit` package connected to your Upstash Redis instance to limit users to, for example, 5 scans per hour. This stops bots from exhausting your AI compute and Upstash daily limits.
- **Automated Data Sanitization (`node-cron`):** You only get 1GB of free storage on Supabase. You will write a cron job in Node.js that runs every hour. It will look for scans with a status of `COMPLETED` and automatically delete their `.zip` files from Supabase Storage. This keeps your storage usage near 0GB and ensures high data privacy.